WebAssembly Execution Modes Under a Real-Time Lens

Davide Saladino
Università degli Studi di Padova
Padova, Italy
davide.saladino@studenti.unipd.it

Valeri Mihail Belenkov
Università degli Studi di Padova
Padova, Italy
valerimihail.belenkov@studenti.unipd.it

Abstract

Studies of WebAssembly on microcontrollers usually report how fast each execution mode is. For real-time use the more relevant question is how predictable it is. We ran four benchmarks natively and under WAMR's classic interpreter, fast interpreter and ahead-of-time compiler, on Zephyr on an RP2350 in both its ARM and RISC-V configurations and on an ESP32-C6 and analysed the results.

1 Introduction

WebAssembly (Wasm) is a portable binary instruction format that was originally designed to run untrusted code safely and at near-native speed in web browsers, and has since extended well beyond the web [3]. Its combination of portability across architectures, a strong sandboxing model, and the ability to serve as a compilation target for languages such as C, C++ and Rust has made it an attractive execution platform for embedded and Internet-of-Things systems, where it is increasingly used to deploy and update application code on resource-constrained devices [13]. On such devices the same Wasm module can be executed in different ways: interpreted directly by a runtime, or first compiled to native machine code ahead of time. These modes trade implementation simplicity and startup behaviour against raw execution speed, and choosing between them is a central design decision when deploying Wasm on a microcontroller.

Embedded systems, however, are frequently also *real-time* systems. In a real-time system, correctness depends not only on the value a computation produces but also on when it is produced [9], so a slower but tightly bounded execution can be worth more than a faster one whose timing varies widely. The pertinent question about a Wasm execution mode is therefore not only how fast it is on average, but how *predictable* it is: how tightly bounded its execution time is, since that bound is what the schedulability of a real-time system ultimately depends upon [1, 11].

Existing studies of WebAssembly on embedded and IoT platforms have largely evaluated execution modes along the axes of average performance and energy consumption, reporting for instance the slowdown of interpreted or compiled Wasm relative to native code [4, 13]. What has received much less attention is the predictability of execution across modes: the distribution of execution times, its variability, and what these imply for worst-case timing and for the schedulability of the resulting system. For real-time use this is arguably the more important axis, and it is the gap this work addresses.

This report reproduces, in the small, the kind of embedded WebAssembly benchmarking reported in prior work [4], using the WebAssembly Micro Runtime (WAMR) on top of the Zephyr real-time operating system. We build on this approach by also considering predictability: alongside average speed, we characterise the execution-time distribution of each mode, and we read the results through the real-time concepts described in Section 2.4. The perimeter of the study is deliberately bounded: four execution modes and four benchmarks, measured on two microcontrollers in three different build configurations.

The remainder of the report is organised as follows. Section 2 introduces WebAssembly, the runtime and RTOS used, and the real-time concepts through which the results are interpreted. Section 3 describes the experimental setup and measurement methodology. Section 4 reports the results, Section 5 interprets them, and Section 6 offers a self-assessment of the work and its limitations.

2 Background

This section introduces the technologies under study and the real-time systems concepts through which we interpret the results.

2.1 WebAssembly

WebAssembly (Wasm) is a portable binary instruction format, originally designed for the web but increasingly adopted beyond it, that serves as a compilation target for languages such as C, C++ and Rust [3]. Its design goals are portability, near-native performance and safe, sandboxed execution.

Execution is specified over an abstract stack machine. Instructions push and pop typed values (32- and 64-bit integers and floats, 128-bit vectors, and references) on an operand stack, rather than operating on named registers. A module's mutable state lives in a *linear memory*: a single contiguous, sandboxed array of bytes, addressed by integer offsets and grown in fixed 64 KB pages. Every access to linear memory is bounds-checked against its current size, and a module can access only its own memory, so an out-of-bounds access is defined to *trap* (a clean, well-defined abort) rather than to produce undefined behaviour [10]. This sandbox is central to WebAssembly's safety guarantees, and, as we discuss later, its enforcement cost and its integrity both depend on the execution mode.

The same Wasm bytecode can be executed in several ways, and this distinction is the subject of our study:

- **Interpretation**: the runtime executes the bytecode directly, fetching, decoding and dispatching each instruction in a software loop. No machine code is generated, so startup is immediate, but every instruction pays a per-instruction dispatch cost on every execution. Runtimes may offer more than one interpreter: WAMR, used in this work, provides both a classic interpreter and a faster interpreter variant that reduces per-instruction overhead at the cost of a larger memory footprint [12].
- **Ahead-of-time (AOT) compilation**: the bytecode is translated to native machine code before execution, in a separate offline step. At run time the processor executes native code directly, eliminating the dispatch loop.

- **Just-in-time (JIT) compilation**: the bytecode is compiled to native code during execution. It attains near-native speed on average, but the compilation work occurs at run time, introducing timing variability. JIT is outside the scope of this study and noted only for completeness.

Interpretation and compilation therefore trade average speed against other properties: an interpreter's per-instruction cost is uniform and its safety checks are enforced by a small, auditable runtime, whereas compiled code runs faster but shifts the enforcement of the sandbox onto the correctness of the generated code [13].

2.2 WebAssembly Micro Runtime

The WebAssembly Micro Runtime (WAMR) is a lightweight runtime maintained under the Bytecode Alliance, designed for embedded and resource-constrained environments [2]. We selected it because it supports all of the execution modes we compare (a classic interpreter, a faster "fast" interpreter, and AOT compilation via its `wamrc` compiler). This allows us to measure differences in the execution mode alone, rather than between separate runtimes with separate code bases [13]. WAMR runs on bare metal and on RTOSes including Zephyr, and targets architectures such as ARM and RISC-V, both of which are tested in this study.

2.3 Zephyr

Zephyr is an open-source real-time operating system for resource-constrained devices [5]. It differs in intent from a general-purpose operating system: where the latter optimises for average throughput and fairness, an RTOS is built to provide predictable, bounded timing behaviour. Its default scheduler is priority-based and preemptive, and thread priorities are static unless the application changes them [5], which is the fixed-priority model under which the response-time analysis of Section 2.4 applies. In our stack Zephyr provides the execution environment beneath WAMR (and beneath the native baseline), on top of the microcontroller hardware, as shown in Figure 1.

2.4 Real-Time Concepts

Since we want to reason about predictability, we need a few concepts from real-time systems, which we introduce here and use throughout the analysis.

Execution time and WCET. The execution time of a task is not a single value: it varies with input data and hardware state. The *worst-case execution time* (WCET) is the upper bound on this time across all conditions. Computing the exact WCET is not generally feasible, so bounds are estimated, and a bound is useful only if it is *safe* (never underestimates) and reasonably *tight* [11]. Bounds may be obtained by static analysis (analysing the program and a hardware model without executing it) or by measurement (running the code and recording the highest observed time). Measurement-based estimates come with an important limitation: the highest observed time is a *high-watermark* and a lower bound on the true WCET, not the WCET itself, since the true worst case may never have been exercised [7, 11].

Response-time analysis. Response-time analysis determines whether a set of tasks meets its deadlines under fixed-priority scheduling. The response time of a task is its own computation time plus interference from higher-priority tasks, and the analysis takes each task's worst-case computation time, denoted C_i, as an *input* [1]. The choice of execution mode changes C_i and its variability, and because C_i appears both in a task's own term and in the interference on lower-priority tasks, a change in C_i propagates through the schedulability of the whole system.

Predictability and jitter. Predictability is the degree to which timing behaviour can be established in advance; a highly predictable system exhibits little variation between executions. That variation is called *jitter*, and the form we measure here is the spread of execution times for a fixed workload.

Mixed-criticality. Modern embedded systems increasingly integrate components of different *criticality* (the severity of the consequences of their failure) onto shared hardware, because dedicating a processor to each component leaves most of that processor idle. What makes the sharing safe is isolation: a low-criticality component must not be able to disturb a high-criticality one. WebAssembly's sandbox provides isolation in software, and how much of it survives depends on the execution mode.

3 Experimental Setup

We compare four execution modes on a set of benchmarks chosen so that each isolates a distinct dimension along which interpreted and compiled execution are expected to differ:

- **Recursive Fibonacci**: near-pure function-call and instruction-dispatch overhead with negligible data, isolating the cost of the interpreter's per-instruction dispatch loop.
- **Matrix multiplication** (64×64, single-precision floating point): floating-point arithmetic over regular, cache-friendly memory access patterns.
- **CRC32** over a 32 KB buffer: a tight integer inner loop (shifts, XOR, a per-bit data-dependent branch) that streams over a large buffer, so that every byte access pays WebAssembly's bounds check. It stresses both dispatch density and memory-access overhead.
- **Quicksort** (4096 32-bit integers): a branch-heavy, recursive algorithm that stresses conditional-branch behaviour. The array is regenerated from a fixed linear-congruential (LCG) sequence before every iteration, outside the measured region, so that each iteration sorts an identical permutation and any measured variability reflects the runtime rather than the input.

For every benchmark we record the CPU cycle count, in each of the following execution modes:

- Native machine code
- Interpreted (classic interpreter)
- Fast interpreted
- Ahead-of-time (AOT) compiled machine code

Measurements were taken on two microcontrollers, in three build configurations:

- `rpi_pico2_thumb`: RP2350, ARM Cortex-M33 core, 150 MHz (Zephyr board `rpi_pico2/rp2350a/m33`).

- `rpi_pico2_riscv32`: the same RP2350 silicon running its RISC-V (Hazard3) core instead, 150 MHz (Zephyr board `rpi_pico2/rp2350a/hazard3`).
- `esp32c6_riscv32`: ESP32-C6, a different micro-controller with a RISC-V core, 16 MHz (Zephyr board `esp32c6_devkitc/esp32c6/hpcore`).

We used Zephyr as the underlying real-time operating system (RTOS) and the WebAssembly Micro Runtime (WAMR) as the WebAssembly runtime, following the architecture shown in Figure 1.

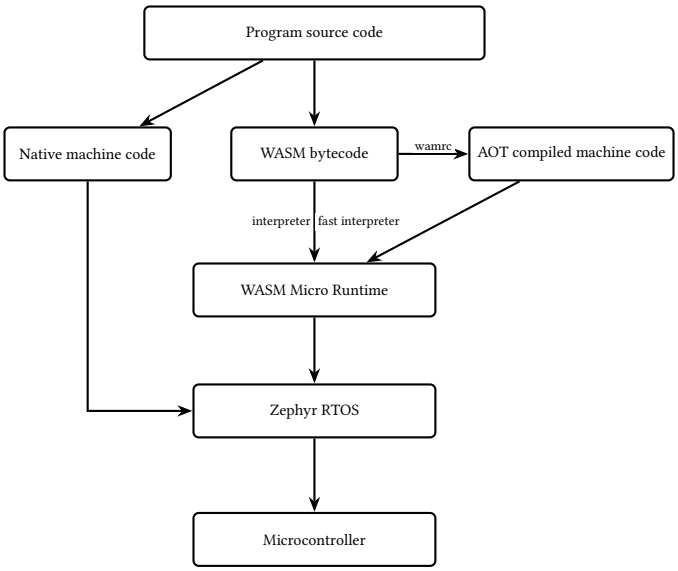

Figure 1: Architecture of the benchmarking environment.

3.1 Measurement Methodology

Each benchmark exports two functions, `bench_init` and `bench_run`. Only `bench_run` is timed. `bench_init` regenerates the input data and runs before every iteration, outside the measured region, so that the figures describe the computation and not its setup. The cycle counter is read with Zephyr's `k_cycle_get_32()` immediately before and after the call, converted to nanoseconds with `k_cyc_to_ns_floor64()`. Each configuration runs for 20 consecutive iterations within a single boot, with a one-second pause between them, and every sample is recorded (without the pause, some output was lost over the serial link). The native baseline uses the same measurement code, with WAMR absent from the build.

Zephyr's cycle counter does not tick at the same rate on every platform. On the RP2350 it follows the 150 MHz CPU clock. On the ESP32-C6 it advances at 16 MHz. Cycle counts are therefore comparable between modes on one platform but not between platforms, where we use elapsed time or dimensionless ratios instead.

The first iteration behaves differently from the rest, and in two distinct ways. In 41 of the 48 configurations it is the largest of the twenty samples, which we read as a warm-up effect. The other seven configurations show the opposite, and six of them are on the RP2350 RISC-V core. Values below the steady state are not physically plausible for a first execution, so we attribute these to the cycle counter not yet being valid rather than to the workload. We therefore report steady-state statistics over iterations 1 to 19 and treat the first sample separately.

For each configuration we report the median, the observed maximum, and the spread between minimum and maximum, with native execution as the baseline for slowdown factors. Following Section 2.4, we call that maximum an observed high-watermark and not a WCET bound. Alongside execution time we report the size of the deployed module.

3.2 Reproducibility

The complete source, build scripts and raw per-sample measurements are available at https://github.com/Valeryum999/RTKS_Assignment. Each benchmark is compiled to WebAssembly with `wasi-sdk 33.0` (clang, `-O3`, `-nostdlib`, an 8 KB shadow stack and a 64 KB initial linear memory, exporting only `bench_init` and `bench_run`), and the module is embedded in the firmware image as a byte array. Ahead-of-time modules are produced from the same `.wasm` file with `wamrc`: for the Cortex-M33 build with `–target=thumbv8m.main –cpu=cortex-m33`, and for both RISC-V builds with `–target=riscv32 –target-abi=ilp32 –cpu=generic-rv32 –cpu-features=+i,+m,+c,-a,-f,-d`. The Cortex-M33 configuration additionally requires `CONFIG_FPU=y` in the Zephyr project configuration; without it the kernel does not grant the FPU access that the compiled module needs. The runtime is WAMR on main branch at commit `646b5a9bc786bfdb69f01e136706d524d4466120` built for the Zephyr platform with the built-in libc and a 128 KB global heap pool, in one of three mutually exclusive configurations: `WAMR_BUILD_INTERP=1` with `WAMR_BUILD_FAST_INTERP=0` (classic interpreter), the same with `WAMR_BUILD_FAST_INTERP=1` (fast interpreter), and `WAMR_BUILD_AOT=1` with `WAMR_BUILD_INTERP=0` (AOT). The native baseline omits WAMR entirely and compiles the same benchmark source directly for the target CPU. Firmware is built and flashed with Zephyr on the main branch at commit `6ab04eeedd2036cdae06538056bc55848f9e5f82` via `west` and OpenOCD, and each run emits CSV rows of the form: `benchmark, mode, platform, iteration, value_cycles, value_ns`.

4 Results

All values reported here derive from the 912 steady-state samples, that is the 960 recorded samples less the first iteration of each configuration, following the treatment described in Section 3. A reported value is the median over iterations 1 to 19 of a configuration. Cycle counts are converted to time for readability and for cross-platform comparability; one cycle is 6.67 ns on the RP2350 and 62.5 ns on the ESP32-C6.

4.1 Execution Time

In all twelve platform-benchmark combinations the four modes fall in the same order: native, then ahead-of-time compilation, then the fast interpreter, then the classic interpreter. Table 1 gives the median execution time of every configuration and Table 2 the corresponding slowdown relative to native execution on the same platform, which Figure 2 presents graphically.

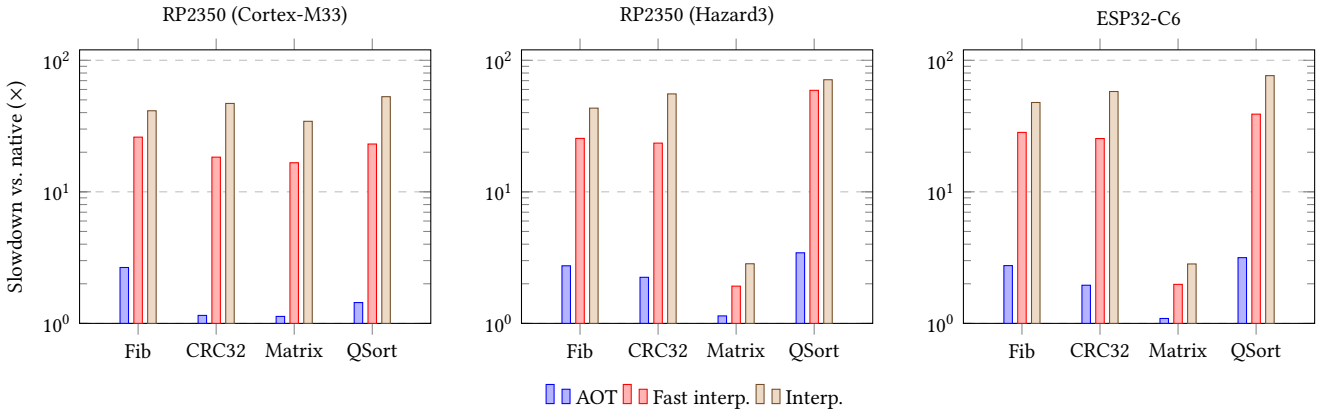

Figure 2: Slowdown relative to native execution on the same platform, logarithmic scale, bars measured from the native baseline at 1×.

Table 1: Median execution time in milliseconds, all 48 configurations.

Mode	Platform	Fib.	CRC32	Matrix	Quick.
Native	RP2350 (Cortex-M33)	167.67	2.19	16.04	4.80
	RP2350 (Hazard3)	171.06	2.18	348.74	4.08
	ESP32-C6	166.49	2.25	374.23	4.08
AOT	RP2350 (Cortex-M33)	446.26	2.52	18.14	6.94
	RP2350 (Hazard3)	468.83	4.90	396.07	14.03
	ESP32-C6	457.44	4.41	407.64	12.87
Fast interp.	RP2350 (Cortex-M33)	4367.36	40.12	266.78	110.94
	RP2350 (Hazard3)	4359.01	51.24	670.25	241.38
	ESP32-C6	4712.23	57.29	740.52	158.87
Interp.	RP2350 (Cortex-M33)	6930.32	102.73	551.16	254.03
	RP2350 (Hazard3)	7401.52	121.28	991.06	290.46
	ESP32-C6	7951.85	130.47	1057.88	311.65

Table 2: Slowdown relative to native execution on the same platform.

Mode	Platform	Fib.	CRC32	Matrix	Quick.
AOT	RP2350 (Cortex-M33)	2.66	1.15	1.13	1.44
	RP2350 (Hazard3)	2.74	2.24	1.14	3.44
	ESP32-C6	2.75	1.95	1.09	3.16
Fast interp.	RP2350 (Cortex-M33)	26.05	18.35	16.64	23.09
	RP2350 (Hazard3)	25.48	23.46	1.92	59.11
	ESP32-C6	28.30	25.42	1.98	38.96
Interp.	RP2350 (Cortex-M33)	41.33	46.98	34.37	52.88
	RP2350 (Hazard3)	43.27	55.51	2.84	71.14
	ESP32-C6	47.76	57.89	2.83	76.43

The penalty for interpretation is uniform across benchmarks on the RP2350 (Cortex-M33) but not on the RP2350 (Hazard3) and the ESP32-C6. On the Cortex-M33, classic interpretation costs between 34.37 and 52.88 times native across all four benchmarks. On both RISC-V platforms three benchmarks fall in a comparable band, between 43.27 and 76.43 times, while matrix multiplication stands apart at 2.84 and 2.83 times. Across all configurations, ahead-of-time compilation costs between 1.09 and 3.44 times native, the fast interpreter between 1.92 and 59.11 times, and the classic interpreter between 2.83 and 76.43 times; the fast interpreter improves on the classic interpreter by a factor between 1.20 and 2.56.

In absolute terms the range is wide. The shortest configuration is CRC32 running natively, at 2.18 ms, and the longest is Fibonacci under the classic interpreter, at 6930 ms on the Cortex-M33 and 7952 ms on the ESP32-C6. Matrix multiplication under interpretation exceeds one second on both RISC-V platforms while taking 551 ms on the Cortex-M33.

4.2 Execution-Time Variability

Every configuration has an observed spread below 0.2% of its median (Table 3). Of the 48, 33 are below 0.01% and 45 below 0.1%; the largest is 0.189%, on the RP2350 (Hazard3) core for Quicksort in native and fast-interpreter modes. In absolute terms on the RP2350 (Cortex-M33) the spread ranges from 1.9 μs (CRC32 under AOT) to 33.0 μs (matrix multiplication under the classic interpreter).

Since the maximum cannot exceed the median by more than the full spread, these figures also bound the distance between the observed maximum and the typical value: at most 0.189% in the worst configuration and at most 0.05% in 43 of the 48. Relative spread does not grow with the interpretation overhead. On the RP2350 (Cortex-M33) the classic interpreter runs up to 52.88 times longer than native yet holds spreads an order of magnitude smaller, and what spread tracks instead is duration, the four widest on that platform belonging to its four shortest configurations and the two narrowest to its two longest.

These figures describe steady-state execution. Because the first iteration is excluded and is, in 41 of the 48 configurations, the largest sample recorded, including it raises the largest excess over the median from 0.19% to 0.62%. An estimate intended to cover the first execution after a cold start would need to be based on that sample rather than on the steady-state high-watermark.

Table 3: Observed spread (maximum minus minimum) as a percentage of the median, all 48 configurations. The ESP32-C6 figures are zero because its 16 MHz counter cannot resolve the variation; see Section 3.

Mode	Platform	Fib.	CRC32	Matrix	Quick.
Native	RP2350 (Cortex-M33)	0.011	0.157	0.031	0.080
	RP2350 (Hazard3)	0.005	0.085	0.001	0.189
	ESP32-C6	0.000	0.000	0.000	0.000
AOT	RP2350 (Cortex-M33)	0.002	0.073	0.017	0.061
	RP2350 (Hazard3)	0.000	0.028	0.001	0.011
	ESP32-C6	0.000	0.000	0.000	0.000
Fast interp.	RP2350 (Cortex-M33)	0.000	0.031	0.005	0.024
	RP2350 (Hazard3)	0.000	0.010	0.002	0.189
	ESP32-C6	0.000	0.000	0.000	0.000
Interp.	RP2350 (Cortex-M33)	0.000	0.014	0.006	0.007
	RP2350 (Hazard3)	0.000	0.009	0.007	0.003
	ESP32-C6	0.000	0.000	0.000	0.000

4.3 Architecture and Device

Natively, the three platforms are closely comparable for three of the four benchmarks: the medians in Table 1 agree within 2% for Fibonacci and within 3% for CRC32, and Quicksort runs 15% faster on both RISC-V platforms than on the Cortex-M33. Matrix multiplication is the exception, and by a wide margin: it takes 21.75 times longer on the RP2350 (Hazard3) core and 23.34 times longer on the ESP32-C6 than on the Cortex-M33, which is the only core in the study with a hardware floating-point unit; on the other two, single-precision arithmetic is emulated in software.

The same division appears in the compiled modes. The AOT median for matrix multiplication is 21.83 times higher on the RP2350 (Hazard3) core than on its Cortex-M33 core, tracking the native ratio of 21.75, whereas under the classic interpreter the two cores differ by only 1.80 times.

The two RISC-V platforms differ in device but not architecture, and their slowdown factors mostly agree for ten of the twelve mode-benchmark pairs.

4.4 Module Size

Ahead-of-time compilation expands the deployed module by a factor between 1.44 and 5.02 (Table 4). The bytecode module is identical across platforms, and the compiled module is target-specific, although the two RISC-V configurations share identical `wamrc` arguments and so yield byte-identical output. The compiled module is larger for RISC-V than for the Cortex-M33 in every case, by 1.06 times for Fibonacci, 1.09 for CRC32, 1.17 for Quicksort and 2.04 for matrix multiplication.

5 Analysis

5.1 Execution Mode and Schedulability

The quantity that response-time analysis consumes is the worst-case computation time C_i of each task [1]. Our measurements show that the choice of execution mode moves this input by a factor of up to 76 for the same source code on the same hardware. Because C_i enters both a task's own response-time term and the interference on every lower-priority task, a change of that magnitude greatly affects the whole task set. A system that is schedulable with ahead-of-time compiled modules may be infeasible under interpretation, with no change whatsoever to the application code.

Table 4: Deployed module size in bytes. The two RISC-V configurations produce identical compiled modules.

		ARM		RISC-V	
Benchmark	`.wasm`	`.aot`	ratio	`.aot`	ratio
Fibonacci	207	980	4.73	1040	5.02
CRC32	1509	2176	1.44	2376	1.57
Matrix mult.	1443	2912	2.02	5928	4.11
Quicksort	589	1620	2.75	1896	3.22

For instance, Fibonacci under the classic interpreter has a C_i of 6930 ms on the Cortex-M33 and 7952 ms on the ESP32-C6. A task that takes seven seconds to run cannot be given a period shorter than seven seconds, which will be infeasible against most real-time requirements; compiled ahead of time the same computation has a C_i of 446 ms, and natively 168 ms. Execution mode is therefore not an implementation detail to be settled after the timing analysis, but one of the inputs that analysis depends on.

5.2 Interpretation Penalty

The penalty for interpretation is not a fixed property of WebAssembly: across our configurations it ranges from 2.83 to 76.43 times native. The reason is that the interpreter's overhead is charged per bytecode instruction, so the penalty depends on how much native work each instruction stands for. Where an instruction represents a cheap operation, the dispatch cost dominates it; where it represents an expensive one, the dispatch cost is small by comparison.

Matrix multiplication exhibits both regimes on the same chip. On the RP2350 Cortex-M33, where single-precision arithmetic runs on the floating-point unit, each multiplication is cheap and the interpretation penalty is 34.37 times, the same order as the three integer benchmarks. On the RP2350's Hazard3 core the same arithmetic is emulated in software, each multiplication is expensive, and the penalty falls to 2.84 times.

The cost of interpretation therefore cannot be quoted as a single number, nor carried over from a published benchmark suite. A workload dominated by expensive operations will appear interpreter-friendly for reasons that have nothing to do with the runtime.

5.3 Compilation Benefit

How much ahead-of-time compilation buys is not a fixed property of the toolchain. It depends on configuration choices that are easy to leave at their defaults, and on the target architecture.

The first concerns the ARM floating-point unit. Compiling with `wamrc` without naming the target CPU produces a module that emulates single-precision arithmetic in software even on a core that has hardware floating point, because the compiler defaults to a generic Thumb v8-M profile with no floating-point features. Inspecting the generated modules confirms this directly: the RISC-V modules contain calls to the software routines `__mulsf3` and `__addsf3`, and

the Cortex-M33 module compiled with `-cpu=cortex-m33` contains none. Left at its default the compiler produced a module that took 223 ms for the matrix benchmark; naming the CPU brought that down to 18 ms, against 16 ms natively.

The second concerns the baseline itself. In an early configuration we measured ahead-of-time compiled Wasm running faster than the native code it was compared against, which is not a plausible result. The cause was that Zephyr defaults to `-Os`, so the native baseline had been optimised for size while `wamrc` defaults to `-O3`. A comparison between execution modes is only meaningful if both sides are compiled with equivalent settings.

Configuration aside, the translation is not architecture-neutral either: for the three integer benchmarks the AOT modules are both slower and larger on the Hazard3 core than on the Cortex-M33, even where the native baselines are equal, so the cost of adopting WebAssembly has to be characterised on the intended architecture.

5.4 Predictability and the Quality of the WCET Lower Bound

Every one of the 48 configurations exhibits an observed spread below 0.2% of its median, 45 of them below 0.1% and 33 below 0.01%, and relative spread does not grow with the interpretation overhead. On this class of target, therefore, all four execution modes are close to equally predictable, and the predictability axis does not discriminate between them. Interpretation is enormously slower but not appreciably less regular.

The premise of worst-case reasoning is that timing varies with hardware and execution state [6], and the variability that makes measurement-based estimation fragile on a general-purpose machine comes from caches, dynamic frequency scaling and competing activity. Our configuration has a fixed clock, a single thread with nothing to preempt it, and inputs regenerated identically before every iteration, so the memory system contributes the same number of hits and misses on every run. That contribution lands in the magnitude of C_i, not in its variance. The small variability that remains depends on how long a configuration runs, not on which mode it uses. On the Cortex-M33 the four widest relative spreads belong to its four shortest configurations. That is what one expects if the cause is an occasional external event of roughly fixed cost.

A high-watermark obtained by measurement is in general only a lower bound on the true WCET and may fall far below it [7, 11]; the gap is widest precisely when execution time is state-dependent. In steady state our observed maximum exceeds the median by less than 0.19% throughout and by less than 0.05% in 43 of the 48 cases, so as a number it is stable and reproducible; including the first execution after reset raises that figure to 0.62%. Our inputs are fixed, so an input-dependent benchmark such as Quicksort may have paths we never triggered, and a single thread never suffers the cache eviction that a preempting task would cause.

5.5 Isolation and Mixed-Criticality

WebAssembly's sandbox provides spatial isolation in software, and the interpreter enforces it within a small runtime whose bounds checks execute as ordinary C; ahead-of-time compilation moves that enforcement into generated native code, enlarging the component that must be correct for the isolation guarantee to hold. The price of keeping enforcement in the interpreter, on the Cortex-M33, is a factor between 34 and 53 in processor capacity. Where a low-criticality component has slack of that order, interpretation buys isolation cheaply; where it does not, the same isolation costs most of the processor.

Mixed-criticality integration is made efficient by budgeting a critical task at the level it normally needs rather than at its full worst case, and letting lower-criticality work use the difference; if the task exceeds that budget the system changes mode and honours the full guarantee [8]. The scheme therefore depends on the high-watermark being a stable value. The consistency reported above, under 0.2% in every configuration, is favourable for exactly that: on this class of target the high-watermark is a usable budget in all four modes. One caveat follows from Section 4: a budget set from the steady-state figure would be exceeded on the first execution after reset, so it should be derived from the cold-start sample instead.

6 Self-Assessment

In this chapter we talk about what we think of our work, how we could have improved it and what we gained from it.

6.1 Achievements

We successfully managed to run and debug various programs on micro-controllers, something neither of us had ever done, starting from a simple program that makes the LED blink to being able to run arbitrary code, compiled in native machine code and in Wasm. We were able to work through various problems that arose during testing, such as:

- Compilation errors occurred in both the Zephyr and WAMR environments, as well as within our code.
- Runtime errors ranging from various SIGSEGV, missing AOT symbols and an inability to read the output via serial due to dropped messages.
- Undefined behaviours, such as the micro-controller clock always returning 0 due to a specific register being overwritten via stack overflow.
- Results inconsistencies, for instance we had a specific configuration in which the AOT compiled code ran faster than the native one, and that was because of Zephyr being compiled by default with `-Os` which favors smaller binary size over performance.

Because of all these problems we had to rerun benchmarks countless times, and every time it seemed like we managed to do it another problem came up shortly thereafter. But in the end we managed to pull through and achieve what we think are solid and rigorous benchmarks, on which we expanded upon by analysing and discussing them in this paper.

6.2 Limitations

The benchmarks are all simple execution tasks enclosed in two small functions, `bench_init` and `bench_run`, which run for a limited amount of time with no interaction and then stop. This is in contrast to real tasks which are supposed to run indefinitely and interact with the real world.

Furthermore, the benchmarks are the only process running in the RTOS, meaning unlike other real threads they don't have to

compete for CPU time and don't suffer the overhead of preemption and cache eviction.

We also wanted to include another micro-controller in the benchmarks, specifically the `ESP32S3_mini` with the Xtensa ISA, however after spending some time trying to run the benchmarks on it we found that, in order to be able to run AOT compiled code we had to patch out some functions in the Zephyr codebase, so we decided against it.

AI Usage Declaration

AI assistance was used in the creation and debugging of the benchmarks, in the analysis of the measurement data and in revising this report.

Acknowledgments

We would like to thank Professor Tullio Vardanega for his guidance and feedback throughout this work.

References

[1] Neil C. Audsley, Alan Burns, Mike F. Richardson, Ken Tindell, and Andy J. Wellings. 1993. Applying New Scheduling Theory to Static Priority Pre-emptive Scheduling. *Software Engineering Journal* 8, 5 (1993), 284–292. doi:10.1049/sej.1993.0034

[2] Bytecode Alliance. 2025. WebAssembly Micro Runtime (WAMR). https://github.com/bytecodealliance/wasm-micro-runtime. Accessed: 12 July 2026.

[3] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. Association for Computing Machinery, 185–200. doi:10.1145/3062341.3062363

[4] Konrad Moron and Stefan Wallentowitz. 2025. Benchmarking WebAssembly for Embedded Systems. *ACM Transactions on Architecture and Code Optimization (TACO)* 22, 3 (2025), 1–21. doi:10.1145/3736169

[5] The Zephyr Project. 2025. Zephyr Project Documentation. https://docs.zephyrproject.org/. Linux Foundation. Version 4.4.1. Accessed: 12 July 2026.

[6] Tullio Vardanega. 2026. Real-Time Kernels and Systems, Topic 1: Introduction. Lecture slides, University of Padua. Academic year 2025/2026. https://www.math.unipd.it/~tullio/RTS/2026/T01.pdf. Accessed: 12 July 2026.

[7] Tullio Vardanega. 2026. Real-Time Kernels and Systems, Topic 10: Worst-Case Execution Time Analysis. Lecture slides, University of Padua. Academic year 2025/2026. https://www.math.unipd.it/~tullio/RTS/2026/T10.pdf. Accessed: 12 July 2026.

[8] Tullio Vardanega. 2026. Real-Time Kernels and Systems, Topic 14: Mixed-Criticality Systems. Lecture slides, University of Padua. Academic year 2025/2026. https://www.math.unipd.it/~tullio/RTS/2026/T14.pdf. Accessed: 12 July 2026.

[9] Tullio Vardanega, Juan Zamorano, and Juan Antonio de la Puente. 2005. On the Dynamic Semantics and the Timing Behavior of Ravenscar Kernels. *Real-Time Systems* (2005). doi:10.1023/B:TIME.0000048937.17571.2b

[10] WebAssembly Community Group. 2025. WebAssembly Core Specification. https://webassembly.github.io/spec/core/. W3C. Accessed: 12 July 2026.

[11] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström. 2008. The Worst-Case Execution-Time Problem—Overview of Methods and Survey of Tools. *ACM Transactions on Embedded Computing Systems (TECS)* 7, 3 (2008), 1–53. doi:10.1145/1347375.1347389

[12] Jun Xu, Liang He, Xin Wang, Wenyong Huang, and Ning Wang. 2021. A Fast WebAssembly Interpreter Design in WASM-Micro-Runtime. Intel Developer Technical Article, ID 671590. https://www.intel.com/content/www/us/en/developer/articles/technical/webassembly-interpreter-design-wasm-micro-runtime.html. Accessed: 12 July 2026.

[13] Yixuan Zhang, Mugeng Liu, Haoyu Wang, Yun Ma, Gang Huang, and Xuanzhe Liu. 2024. Research on WebAssembly Runtimes: A Survey. *ACM Transactions on Software Engineering and Methodology* (2024). doi:10.1145/3714465